

Übungsblatt 3

Frontend-Technologien

5430UE Web and Data Engineering

24. März 2026

Prof. Dr. Michael Granitzer

Lernziele

- **Semantisches HTML** strukturieren und Barrierefreiheit (a11y) berücksichtigen
- **Personas und Nutzerszenarien** für Frontend-Entscheidungen einsetzen
- **CSS-Layouts** mit Flexbox/Grid und **responsive Design** mit Media Queries umsetzen
- **JavaScript** für DOM-Manipulation, fetch und asynchrone Verarbeitung verwenden
- Den Unterschied zwischen **statischem**, **server-rendertem** und **clientseitigem** Frontend einordnen

Modul A — HTML

Übung 3.A.1: Semantisches HTML

Gegeben ist folgendes unsemantisches HTML:

```
<div class="header">
  <div class="nav">
    <div class="nav-item"><a href="/">Home</a></div>
    <div class="nav-item"><a href="/products">Produkte</a></div>
  </div>
</div>
<div class="main">
  <div class="article">
    <div class="title">Produktname</div>
    <div class="text">Beschreibung...</div>
  </div>
</div>
<div class="footer">© 2026</div>
```

1. Schreiben Sie die Struktur mit semantischen HTML5-Elementen um.
2. Warum ist semantisches HTML für Suchmaschinen und Screenreader wichtig?

Übung 3.A.1: Semantisches HTML — Lösung

```
<header>
  <nav>
    <a href="/">Home</a>
    <a href="/products">Produkte</a>
  </nav>
</header>
<main>
  <article>
    <h1>Produktname</h1>
    <p>Beschreibung...</p>
  </article>
</main>
<footer>© 2026</footer>
```

Warum semantisch?

- **Suchmaschinen:** Crawler verstehen die Bedeutung der Inhalte — `<h1>` signalisiert Hauptüberschrift, `<nav>` Navigationsbereiche. Das verbessert SEO.
- **Screenreader:** Blinde Nutzer navigieren per Tastatur durch Landmarks (header, main, nav). Ohne Semantik ist jedes `<div>` gleichwertig — Navigation wird unmöglich.

Übung 3.A.2: Formulare und Accessibility

Erstellen Sie ein barrierefreies HTML-Formular für eine Produktsuche mit:

- Texteingabe für den Suchbegriff (Pflichtfeld)
- Dropdown für Kategorie (Elektronik, Kleidung, Bücher)
- Checkbox „Nur verfügbare Produkte“
- Submit-Button

Welche drei HTML-Attribute sind für Barrierefreiheit unverzichtbar?

Übung 3.A.2: Formulare und Accessibility — Lösung

```
<form action="/search" method="GET">
  <label for="query">Suchbegriff</label>
  <input id="query" name="query" type="text" required
    placeholder="z.B. Laptop" aria-required="true"/>

  <label for="category">Kategorie</label>
  <select id="category" name="category">
    <option value="">Alle</option>
    <option value="electronics">Elektronik</option>
    <option value="clothing">Kleidung</option>
    <option value="books">Bücher</option>
  </select>

  <label>
    <input type="checkbox" name="available" value="true"/>
    Nur verfügbare Produkte
  </label>

  <button type="submit">Suchen</button>
</form>
```

Unverzichtbare Attribute:

1. **for / id-Verknüpfung:** Verbindet Label mit Input — Klick auf Label fokussiert das Feld.
2. **required:** Zeigt Pflichtfelder an und verhindert leere Submits (Browser-Validierung).
3. **type (auf <input>):** Bestimmt Eingabemodus und Tastaturlayout auf Mobilgeräten.

Übung 3.A.3: Barrierefreiheit — Problemkategorien und Prinzipien

Barrierefreiheit ist nach EU-Richtlinien ein verpflichtender Aspekt der Web-Entwicklung. Barrieren werden in drei Kategorien klassifiziert: **Wahrnehmungs-**, **Verständnis-** und **Zugriffsprobleme**.

1. Nennen und erklären Sie zu jeder Kategorie zwei Beispiele. Welche Kategorie ist hauptsächlich technischer Natur?
2. Was verbirgt sich hinter dem **Selbstbestimmungsprinzip**, dem **Zwei-Sinne-Prinzip** und dem **Universal-Design-Prinzip**?
3. Nennen Sie ein Prüftool für Barrierefreiheit. Ist die Schriftfarbe `#ffffff` auf dem Hintergrund `#0aae00` barrierefrei?

Übung 3.A.3: Barrierefreiheit — Lösung

1. Problemkategorien

Kategorie	Beispiele
Wahrnehmung	Ungenügende Farbkontraste (Rotgrünblindheit, gelbe Schrift auf weiß); mangelnde Skalierbarkeit (kleine statische Texte, Bilder mit fester niedriger Auflösung)
Verständnis	Ungenügende Semantik (Überschriften nur durch Schriftart, Screenreader erkennt sie nicht); Informationsüberfluss; keine Trennung leichte/komplexe Sprache
Zugriff	Zu knappe Timeouts (Session läuft ab vor Submit); Eingabegerät-Abhängigkeit (Bedienung nur mit Maus, keine Tastaturnavigation)

Zugriffsprobleme sind hauptsächlich technischer Natur — sie ließen sich durch universellen Zugang zu Hardware und Bandbreite weitgehend beheben.

2. Drei Prinzipien



- **Selbstbestimmungsprinzip:** Die Website bietet flexible Darstellungen — Hochkontrast-Schema, Schriftgrößendynamik. Nutzende konfigurieren die Seite nach eigenen Bedürfnissen.
- **Zwei-Sinne-Prinzip:** Jede Information wird über mindestens zwei verschiedene Sinne angeboten (z.B. Bildschirm + Screenreader-fähig).
- **Universal-Design-Prinzip:** Keine separaten Zugänge für Menschen mit Behinderung — alles wird über ein einheitliches System ausgeliefert (Wartbarkeit + Übersicht).

3. Prüftool und Beispiel

Werkzeug: **WAVE Browser Plugin** (wave.webaim.org/extension).

Für #ffffff auf #0aae00: Kontrastverhältnis $\approx 2.97:1$ — fällt sowohl bei WCAG AA als auch AAA durch. **Nicht barrierefrei.**

Übung 3.A.4: Personas — Entertainmentsystem im Flugzeug

Sie unterstützen einen Flugzeughersteller bei der Entwicklung eines In-Flight-Entertainmentsystems mit Touchscreens in den Sitzlehnen der ersten Klasse. Diskutiert werden verschiedene Bedienmethoden (Touch, Tastaturpad, Armlehnenarmatur). Auch unterschiedliche technische Affinität von Passagieren und Personal soll berücksichtigt werden.

1. Was ist eine **Persona** im Kontext der Anforderungserhebung, und welche Felder sollte sie typischerweise enthalten?
2. Entwerfen Sie eine **primäre** und eine **sekundäre** Persona für das System.
3. Beschreiben Sie für jede Persona ein **Szenario**, in dem sie das Entertainmentsystem benutzt.

Übung 3.A.4: Personas — Lösung

1. Persona = fiktive, aber repräsentative Benutzerfigur einer Zielgruppe. Typische Felder: Name, Alter, Wohnort, Bildung/Beruf, technische Affinität, Ziele, Hindernisse, eine kurze Biografie.

2. Primäre Persona — Günther Vogt (69, Erlangen)

Bäcker im Ruhestand, besitzt nur ein altes Smartphone, geringe technische Affinität. Reist gerne, lässt sich von Naturdokumentationen inspirieren. Möchte unabhängig bleiben, hat aber durch Arthritis Schwierigkeiten mit kleinen Touch-Bedienelementen.

Sekundäre Persona — Stefanie Sauer (37, München)

Fluggerätmechanikerin (Lufthansa Technik), spezialisiert auf Messtechnik. Hohe technische Affinität, im Beruf Wartung der Innenraumelektronik — also auch des Entertainmentsystems. Hat private Bluetooth-Kopfhörer und erwartet Standardkompatibilität.

3. Szenarien



- **Günther** auf dem Heimflug aus Portugal: möchte Naturdokumentationen finden, bereits besuchte Länder ausfiltern, große sortierbare Liste mit großen Bedienflächen, Pause-Funktion mit einem Tastendruck wenn das Essen serviert wird.
- **Stefanie** als Passagierin nach Ibiza: koppelt ihre AirPods per Bluetooth, sucht aktuelle Charts, regelt Lautstärke mit max. zwei Interaktionen (initial leise als Hörschutz).
- **Stefanie** im Wartungsmodus: identifiziert sich per RFID-Betriebsausweis, ruft technische Diagnose-Daten ab, ohne durch Nutzungsstatistiken gestört zu werden — alles unter Zeitdruck.

Erkenntnis für das Design: Touch-Bedienung allein reicht nicht — physische Tasten und große Trefferflächen sind nötig (Günther), und ein separater Servicemodus mit RFID-Auth muss vorgesehen werden (Stefanie).

Modul B — CSS

Übung 3.B.1: Box-Modell und Selektoren

Gegeben folgendes CSS:

```
.card {  
  width: 300px;  
  padding: 20px;  
  border: 2px solid black;  
  margin: 16px;  
}
```

1. Wie breit ist `.card` im Standard-Box-Modell (content-box)? Begründen Sie.
2. Wie breit wäre `.card` mit `box-sizing: border-box`?
3. Schreiben Sie einen CSS-Selektor, der nur das erste `.card`-Element innerhalb eines `<section>` trifft.

Übung 3.B.1: Box-Modell und Selektoren — Lösung

1. **Standard** (content-box): `width` gilt nur für den Content-Bereich. Gesamtbreite = $300 + 20 + 20 + 2 + 2 = 344 \text{ px}$. Margin gehört nicht zur Elementbreite, aber zum benötigten Platz im Layout.
2. **border-box**: `width: 300px` ist das Gesamtmaß inklusive Padding und Border. Content-Breite = $300 - 40 - 4 = 256 \text{ px}$. Die Gesamtbreite des Elements bleibt **300 px**.

3.

```
section .card:first-child { }  
/* oder präziser: */  
section > .card:first-of-type { }
```

Übung 3.B.2: Responsive Layout mit Flexbox

Implementieren Sie ein responsives Produktraster:

- Desktop ($\geq 768\text{px}$): 3 Spalten nebeneinander, gleichmäßig verteilt
- Mobile ($< 768\text{px}$): 1 Spalte

Verwenden Sie ausschließlich Flexbox (kein Grid). Schreiben Sie das vollständige CSS für den Container `.product-grid` und die Karten `.product-card`.

Übung 3.B.2: Responsive Layout mit Flexbox — Lösung

```
.product-grid {  
  display: flex;  
  flex-wrap: wrap;  
  gap: 1rem;  
}  
  
.product-card {  
  flex: 1 1 calc(33.33% - 1rem);  
  min-width: 0; /* verhindert Overflow bei langen Texten */  
  box-sizing: border-box;  
}  
  
@media (max-width: 767px) {  
  .product-card {  
    flex: 1 1 100%;  
  }  
}
```

`flex: 1 1 calc(33.33% - 1rem)` bedeutet: `grow=1`, `shrink=1`, Basisbreite $\approx$ ein Drittel minus halber Gap. `flex-wrap: wrap` erlaubt Zeilenumbruch. Auf Mobile wird die Basis auf 100% gesetzt $\rightarrow$ eine Karte pro Zeile.

Übung 3.B.3: CSS-Selektoren lesen und schreiben

Gegeben ist folgendes HTML einer TODO-Liste:

```
<h1>Aufgaben:</h1>
<p>Folgende Aufgaben müssen <strong>unbedingt</strong> erledigt werden.</p>
<ul>
  <li class="styles" for="basic" id="entry01">Aufräumen</li>
  <li>Einkaufen</li>
  <li for="basicStudies">Lernen
    <ol>
      <li id="entry02">HTML</li>
      <li>CSS</li>
    </ol>
  </li>
  <li for="peace">Entspannen</li>
</ul>
```

Schreiben Sie CSS-Regeln für folgende Anforderungen (jeweils ein Selektor):

1. Alle Listeneinträge **ohne** die Klasse `styles` bekommen Schriftgröße 18 px.
2. Das **zweite** Element jeder Liste (TODO und Unterliste „Lernen“) wird ausgeblendet.
3. Der Text „unbedingt“ wird rot gefärbt.
4. Der „Aufgaben:“-Header wird um 10° an der Z-Achse gedreht und um 20 px nach rechts und 40 px nach unten verschoben.
5. Alle Listeneinträge mit `for="basic"` werden durchgestrichen.
6. Wenn der Bildschirm schmaler als 600 px ist, wird die Schriftgröße im body auf 10 px reduziert.

Übung 3.B.3: CSS-Selektoren — Lösung

```
/* 1) Negation: alles außer .styles */
li:not(.styles) { font-size: 18px; }

/* 2) Strukturpseudoklasse: zweites Kind */
li:nth-child(2) { display: none; }

/* 3) Element-Selektor */
strong { color: red; }

/* 4) Zwei Transformationen kombiniert */
h1 { transform: rotateZ(10deg) translate(20px, 40px); }

/* 5) Attributselektor */
li[for="basic"] { text-decoration: line-through; }

/* 6) Media Query */
@media screen and (max-width: 599px) {
  body { font-size: 10px; }
}
```

Konzepte im Überblick:

- `:not(...)` — Negation, schließt Elemente vom Match aus
- `:nth-child(n)` — wählt das n-te Geschwister-Element
- `[attr="value"]` — Attributselektor, prüft auf konkreten Wert
- `transform` — Mehrere Transformationen werden in einer Liste angegeben (Reihenfolge zählt!)
- `@media` — Bedingung an das Ausgabemedium / Viewport

Modul C — JavaScript

Übung 3.C.1: DOM-Manipulation und Events

Implementieren Sie folgende Funktion in JavaScript:

Ein Button `#toggle-theme` soll beim Klick die Klasse `dark-mode` auf dem `<body>` togglen. Zusätzlich soll der Button-Text zwischen „Dark Mode“ und „Light Mode“ wechseln.

Schreiben Sie den vollständigen JavaScript-Code (kein Framework).

Übung 3.C.1: DOM-Manipulation und Events — Lösung

```
const btn = document.getElementById('toggle-theme');

btn.addEventListener('click', () => {
  document.body.classList.toggle('dark-mode');
  const isDark = document.body.classList.contains('dark-mode');
  btn.textContent = isDark ? 'Light Mode' : 'Dark Mode';
});
```

Warum `classList.toggle`? Es fügt die Klasse hinzu, falls sie fehlt, und entfernt sie, falls sie vorhanden ist — in einer Operation, ohne manuelle Prüfung.

Übung 3.C.2: Fetch API und asynchrones JavaScript

Implementieren Sie eine Funktion `loadProduct(id)`, die:

1. Die URL `https://api.example.com/products/{id}` aufruft (GET)
2. Das JSON-Ergebnis in ein `<div id="product-detail">` rendert (Felder: `name`, `price`, `description`)
3. Fehler (Netzwerk oder HTTP-Status ≥ 400) mit einer Fehlermeldung im gleichen `<div>` anzeigt

Verwenden Sie `async/await`.

Übung 3.C.2: Fetch API und asynchrones JavaScript — Lösung

```
async function loadProduct(id) {
  const container = document.getElementById('product-detail');
  try {
    const response = await fetch(`https://api.example.com/products/${id}`);
    if (!response.ok) {
      throw new Error(`HTTP ${response.status}: ${response.statusText}`);
    }
    const product = await response.json();
    container.innerHTML = `
      <h2>${product.name}</h2>
      <p><strong>Preis:</strong> €${product.price.toFixed(2)}</p>
      <p>${product.description}</p>
    `;
  } catch (error) {
    container.innerHTML = `<p class="error">Fehler: ${error.message}</p>`;
  }
}
```

Wichtig: `response.ok` prüft HTTP-Status 200–299. `fetch` wirft bei Netzwerkfehlern eine Exception, aber *nicht* bei HTTP 4xx/5xx — daher die manuelle `if (!response.ok)` Prüfung.

Modul D — Praktische JavaScript- Aufgaben

Übung 3.D.1: Luhn-Algorithmus — Kreditkarten-Validierung

Der **Luhn-Algorithmus** prüft, ob eine Kreditkartennummer formal korrekt ist (Tipp-fehler-Erkennung).
Algorithmus:

1. Letzte Ziffer = Prüfziffer, nicht verändern
2. Von rechts: jede zweite Ziffer verdoppeln
3. Wenn das Ergebnis ≥ 10 : beide Stellen addieren (z.B. $16 \rightarrow 1+6 = 7$)
4. Alle Ziffern aufsummieren
5. Wenn die Summe durch 10 teilbar ist $\rightarrow$ gültig

Implementieren Sie eine JavaScript-Funktion `isValidLuhn(number)`, die:

- Eine Kreditkartennummer als String entgegennimmt
- `true` zurückgibt wenn die Nummer den Luhn-Check besteht, sonst `false`

Testen Sie mit: 4532015112830366 (gültig) und 1234567890123456 (ungültig)



Übung 3.D.1: Luhn-Algorithmus — Kreditkarten-Validierung — Lösung

```
function isValidLuhn(number) {  
  const digits = number.split('').map(Number).reverse();  
  let sum = 0;  
  for (let i = 0; i < digits.length; i++) {  
    let d = digits[i];  
    if (i % 2 === 1) {           // jede zweite Stelle (von rechts, 0-indexiert)  
      d *= 2;  
      if (d >= 10) d = Math.floor(d / 10) + (d % 10);  
    }  
    sum += d;  
  }  
  return sum % 10 === 0;  
}  
  
console.log(isValidLuhn('4532015112830366')); // true  
console.log(isValidLuhn('1234567890123456')); // false
```

Erklärung:

- `reverse()` vereinfacht das "von rechts zählen": Index 0 = Prüfstelle (nicht verdoppeln), Index 1, 3, 5, ... werden verdoppelt.
- Verdopplung ≥ 10 : Quersumme = $\text{Math.floor}(d/10) + d\%10$ (äquivalent zu -9).
- Einsatz: Bankformulare validieren Kreditkartennummern **clientseitig** mit Luhn, bevor der Request gesendet wird — reduziert Serverlast und verbessert UX.

Übung 3.D.2: Web Workers und Local Storage

1. **Web Workers:** Ein Online-Shop berechnet Produktempfehlungen per komplexem Scoring-Algorithmus. Nach der Implementierung friert die UI kurz ein. Erklären Sie: a) Warum friert die UI ein? b) Welche Technologie löst dieses Problem, und wie funktioniert die Kommunikation zwischen UI und dem ausgelagerten Code?
2. **Lokale Datenspeicherung:** Ein Nutzer füllt ein mehrstufiges Registrierungsformular aus. Er schließt versehentlich den Tab. Nennen Sie zwei JavaScript-APIs für clientseitige Persistenz und vergleichen Sie:
 - Lebensdauer (bis wann gehen Daten verloren?)
 - Speicherkapazität (ungefähr)
 - Typischer Einsatz

Übung 3.D.2: Web Workers und Local Storage — Lösung

1. Web Workers

a) **Ursache:** JavaScript ist single-threaded. Lange synchrone Berechnungen blockieren den einzigen Thread — dadurch können keine UI-Events (Klicks, Scrolling) verarbeitet werden.

b) **Lösung: Web Worker** — führt JavaScript in einem separaten Thread aus. Kommunikation über `postMessage` (asynchron):

```
// main.js
const worker = new Worker('scoring.js');
worker.postMessage({ products: productList });
worker.onmessage = (e) => renderRecommendations(e.data);

// scoring.js (Web Worker)
self.onmessage = (e) => {
  const result = computeScoring(e.data.products); // rechenintensiv
  self.postMessage(result);
};
```

2. Clientseitige Persistenz

	sessionStorage	localStorage
Lebensdauer	Tab wird geschlossen → Daten weg	Bleibt erhalten (auch nach Browser-Neustart)
Kapazität	~5 MB	~5–10 MB
Typischer Einsatz	Temporärer Formularzustand pro Session	Nutzereinstellungen, Warenkorb-Entwurf



Für den Formular-Zwischenstand: localStorage — Daten überleben das Schließen des Tabs:

```
// Speichern
localStorage.setItem('formStep2', JSON.stringify(formData));
// Laden
const saved = JSON.parse(localStorage.getItem('formStep2'));
```

Übung 3.D.3: Währungsrechner mit Fetch und exchangerate.host

Entwickeln Sie eine HTML-Seite, die Beträge von Euro in andere Währungen umrechnet:

- Eingabefeld für den **Betrag** in Euro
- Auswahlmenü für die **Zielwährung** (USD, CHF)
- Ein „Umrechnen“-Button, der das Ergebnis in einem <output>-Element anzeigt
- Live-Wechselkurse von api.exchangerate.host (kostenlose REST-API)
- Das letzte Ergebnis soll im `localStorage` persistiert und beim erneuten Laden angezeigt werden

Welche **Caching-Strategie** für die Wechselkurse halten Sie für sinnvoll? Begründen Sie.

Übung 3.D.3: Währungsrechner — Lösung

```
<!DOCTYPE html>
<html lang="de">
<body>
  <label for="amount">Betrag in EUR</label>
  <input id="amount" type="number" min="0" step="0.01" value="100"/>

  <label for="target">Zielwährung</label>
  <select id="target">
    <option value="USD">US-Dollar (USD)</option>
    <option value="CHF">Schweizer Franken (CHF)</option>
  </select>

  <button id="convert">Umrechnen</button>
  <output id="result"></output>

  <script>
    const out = document.getElementById('result');
    const last = localStorage.getItem('lastResult');
    if (last) out.textContent = `Letztes Ergebnis: ${last}`;

    document.getElementById('convert').addEventListener('click', async () => {
      const amount = parseFloat(document.getElementById('amount').value);
      const to = document.getElementById('target').value;
      const url = `https://api.exchangerate.host/latest?base=EUR&symbols=${to}`;
      try {
        const r = await fetch(url);
        const data = await r.json();
        const rate = data.rates[to];
```

Caching-Strategie: Wechselkurse ändern sich höchstens minütlich, oft nur einmal pro Tag. Eine **kurze Cache-Zeit** (z.B. Cache-Control: max-age=300 für 5 Minuten) reduziert API-Aufrufe drastisch ohne signifikanten Genauigkeitsverlust. Für nicht-kritische Anwendungen genügt sogar Tages-Cache.

Materialien zum Übungsblatt

Zum Üben mit den HTML/CSS/JS-Aufgaben stehen vorbereitete Vorlagen bereit:

- [luhn-raw.html](#) — leeres Gerüst für die Luhn-Prüfziffer
- [luhn.html](#) + [luhn.js](#) — funktionierende Referenzlösung
- [currency-api-empty.html](#) — Startgerüst für den Währungsrechner
- [currency-api.html](#) — vollständiger Währungsrechner mit fetch
- [media-query.html](#) — Beispiel für responsives Layout
- [exercise-04.html](#) — kombinierte HTML/JS-Übung
- [datenuebertragung.png](#), [persona.png](#) — Abbildungen aus den Folien

*Alle Materialien finden Sie auch zentral auf der **Kursseite** unter Materials → Download.*